

```
// ===== lib/main.dart =====

import 'dart:async';
import 'dart:ui';

import 'package:audio_service/audio_service.dart';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:liquid_glass_widgets/liquid_glass_widgets.dart';

import 'app.dart';
import 'src/core/app_logger.dart';
import 'src/core/application_logger.dart';
import 'src/core/platform_caps.dart';
import 'src/core/rust_init.dart';
import 'src/core/settings.dart';
import 'src/player/cast_provider.dart';
import 'src/plugin/plugin_updates.dart';
import 'src/auth/account_api.dart';
import 'src/player/ios_widget_bridge.dart';
import 'src/player/player_provider.dart';
import 'src/player/player_widget_bridge.dart';
import 'src/deeplink/deep_link_handler.dart';
import 'src/lyrics/floating_lyrics.dart';
import 'src/lyrics/status_bar_lyrics.dart';
import 'src/navigation/routes.dart';
import 'src/watch_link/watch_link_provider.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final imageCache = PaintingBinding.instance.imageCache;
  imageCache.maximumSize = 800;
  imageCache.maximumSizeBytes = 120 << 20;
  final container = ProviderContainer();
  ApplicationLogManager.instance.bootstrap();
  WidgetsBinding.instance
    .addObserver(AppLogLifecycleObserver());
  WidgetsBinding.instance
    .addObserver(AppLogBackGestureObserver());
  BackGestureNativeBridge.init();
  _installErrorReporting(container);
  AppLog.info('startup', '应用启动 (main 开始)');

  container.read(rustInitProvider);

  unawaited(LiquidGlassWidgets.initialize().catchError((Object _) {}));

  runApp(
    LiquidGlassWidgets.wrap(
      adaptiveQuality: true,
      child: UncontrolledProviderScope(
```

```
        container: container,
        child: const AppWarmupRunner(
          child: XianYuApp(),
        ),
      ),
    ),
  );

XianYuDeepLink.init(container, appRouter);

if (PlatformCaps.supportsFloatingLyrics) {
  container.read(floatingLyricsControllerProvider).init();
}

if (PlatformCaps.supportsStatusBarLyrics) {
  container.read(statusBarLyricsControllerProvider).init();
}

if (PlatformCaps.supportsHomeWidgets) {
  container.read(playerWidgetControllerProvider).init();
}

if (PlatformCaps.supportsLiveActivity) {
  container.read(iosWidgetControllerProvider).init();
}

if (PlatformCaps.isAndroid) {
  container.read(watchLinkControllerProvider).init();
}

final t0 = Stopwatch()..start();
WidgetsBinding.instance.addPostFrameCallback((_) {
  final ms = t0.elapsedMilliseconds;
  AppLog.info('startup', '首帧渲染完成 ${ms}ms (从 main 起算)');
});

unawaited(
  AudioService.init(
    builder: () => XianYuAudioHandler(),
    config: const AudioServiceConfig(
      androidNotificationChannelId: 'cc.xymusic.mobile.channel.audio',
      androidNotificationChannelName: '弦予音乐播放控制',

      androidNotificationOngoing: false,
      androidStopForegroundOnPause: false,
      androidNotificationIcon: 'drawable/ic_notification',
      artDownscaleWidth: 512,
      artDownscaleHeight: 512,
      androidNotificationClickStartsActivity: true,
    ),
  ),
);
```

```

    ).then((h) {
      audioHandler = h;
      final notifier = activePlayerNotifier;
      if (notifier != null) h.bindNotifier(notifier);
      AppLog.info('startup', 'AudioService 初始化完成');
    }, onError: (Object e, StackTrace st) {
      AppLog.error('startup', 'AudioService 初始化失败: $e\n$st');
    }),
  );
}

```

```

void _installErrorReporting(ProviderContainer container) {
  var reportingError = false;
  FlutterError.onError = (details) {
    if (reportingError) return;
    reportingError = true;
    scheduleMicrotask(() => reportingError = false);
    final msg = details.exceptionAsString();
    final stack = details.stack?.toString() ?? '';
    AppLogger.instance
      .log('fatal', '未捕获异常: $msg\n$stack');
    AppLog.error('fatal', '$msg\n$stack');
    FlutterError.presentError(details);
    try {
      container.read(accountApiProvider).reportError(
        errorType: 'flutter',
        errorMessage: msg,
        errorStack: stack,
        page: 'global',
      );
    } catch (_) {}
  };
  PlatformDispatcher.instance.onError = (error, stack) {
    if (reportingError) return true;
    reportingError = true;
    scheduleMicrotask(() => reportingError = false);
    AppLogger.instance
      .log('fatal', '平台异常: $error\n$stack');
    AppLog.error('platform', '$error\n$stack');
    try {
      container.read(accountApiProvider).reportError(
        errorType: 'platform',
        errorMessage: error.toString(),
        errorStack: stack.toString(),
        page: 'global',
      );
    } catch (_) {}
    return true;
  };
}

```

```

}

class AppWarmupRunner extends ConsumerStatefulWidget {
  const AppWarmupRunner({super.key, required this.child});

  final Widget child;

  @override
  ConsumerState<AppWarmupRunner> createState() => _AppWarmupRunnerState();
}

class _AppWarmupRunnerState extends ConsumerState<AppWarmupRunner> {
  @override
  void initState() {
    super.initState();

    ref.listenManual(rustInitProvider, (prev, next) {
      if (next.hasValue || next.hasError) _runStartupPluginAutoUpdate();
    });

    ref.listenManual(settingsProvider, (prev, next) {
      if (next.hasValue) {
        final container = ProviderScope.containerOf(context);
        unawaited(container
          .read(dlnaCastProvider.notifier)
          .applyRendererSetting());
      }
    });
  }

  void _runStartupPluginAutoUpdate() {
    runPluginAutoUpdateOnStartup(
      ProviderScope.containerOf(context),
      (message) => AppLogger.instance.log('plugin', message),
    );
  }

  @override
  Widget build(BuildContext context) => widget.child;
}

// ===== lib/app.dart =====
import 'dart:async';
import 'dart:io';

import 'package:flutter/cupertino.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter_localizations/flutter_localizations.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

```

```
import 'src/core/rust_init.dart';
import 'src/core/settings.dart';
import 'src/core/app_colors.dart';
import 'src/auth/account_api.dart';
import 'src/il8n/il8n.dart';
import 'src/navigation/routes.dart';
import 'src/update/app_update.dart';
import 'src/widgets/flying_cover.dart';
import 'src/widgets/privacy_policy.dart';
import 'src/widgets/custom_background.dart';
import 'src/widgets/liquid_wave.dart';
import 'l10n/gen/app_localizations.dart';

const SnackBarThemeData _toastTheme = SnackBarThemeData(
  behavior: SnackBarBehavior.floating,
  width: 240,
  shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.all(Radius.circular(60)),
  ),
  backgroundColor: Color(0xE6323232),
  elevation: 0,
  contentTextStyle: TextStyle(
    color: Colors.white,
    fontSize: 13.5,
    height: 1.3,
    decoration: TextDecoration.none,
  ),
  insetPadding: EdgeInsets.symmetric(vertical: 14),
);

class XianYuApp extends ConsumerStatefulWidget {
  const XianYuApp({super.key});

  @override
  ConsumerState<XianYuApp> createState() => _XianYuAppState();
}

class _XianYuAppState extends ConsumerState<XianYuApp> with WidgetsBindingObserver {
  int? _cachedAccent;
  bool? _cachedPredictiveBack;
  WallpaperTextColor _cachedTextMode = WallpaperTextColor.follow;
  ThemeData? _lightTheme;
  ThemeData? _darkTheme;
  bool _loggedHomeFirstFrame = false;

  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addObserver(this);
  }
}
```

```

@override
void dispose() {
    WidgetsBinding.instance.removeObserver(this);
    super.dispose();
}

@override
void didChangeLocales(List<Locale>? locales) {
    final settings = ref.read(settingsProvider).valueOrNull;
    if ((settings?.language ?? AppLanguage.system) == AppLanguage.system) {
        setState(() {});
    }
}

Future<void> _runStartupAfterConsent(WidgetRef ref) async {

    final navContext = appNavigatorKey.currentContext;
    if (navContext == null) {

        WidgetsBinding.instance.addPostFrameCallback((_) {
            unawaited(_runStartupAfterConsent(ref));
        });
        return;
    }
    final agreed = await ensurePrivacyConsent(navContext);
    if (!agreed || !mounted) return;
    ref.read(accountApiProvider).reportAppOpen();
    unawaited(runStartupVersionChecks(ref));
}

ColorScheme _schemeWithExactAccent(
    {required Color accent, required Brightness brightness}) {
    final dark = brightness == Brightness.dark;
    final base =
        ColorScheme.fromSeed(seedColor: accent, brightness: brightness);
    final hsl = HSLColor.fromColor(accent);
    var primary = accent;
    if (dark && hsl.lightness < 0.4) {
        primary = hsl.withLightness(0.5).toColor();
    }
    Color onOf(Color c) => c.computeLuminance() > 0.55
        ? const Color(0xFF1F1F1F)
        : Colors.white;
    final primaryContainer = dark
        ? Color.lerp(primary, Colors.black, 0.55)!
        : Color.lerp(primary, Colors.white, 0.85)!;
    final onPrimaryContainer = dark
        ? Color.lerp(primary, Colors.white, 0.8)!
        : Color.lerp(primary, Colors.black, 0.45)!;

```

```

    final secondary = hsl
        .withSaturation((hsl.saturation * 0.45).clamp(0.0, 1.0))
        .toColor();
    final secondaryContainer = dark
        ? Color.lerp(secondary, Colors.black, 0.5)!
        : Color.lerp(secondary, Colors.white, 0.85)!;
    final onSecondaryContainer = dark
        ? Color.lerp(secondary, Colors.white, 0.75)!
        : Color.lerp(secondary, Colors.black, 0.4)!;
    return base.copyWith(
        primary: primary,
        onPrimary: onOf(primary),
        primaryContainer: primaryContainer,
        onPrimaryContainer: onPrimaryContainer,
        inversePrimary: dark ? onPrimaryContainer : primary,
        secondary: secondary,
        onSecondary: onOf(secondary),
        secondaryContainer: secondaryContainer,
        onSecondaryContainer: onSecondaryContainer,
        tertiary: primary,
        onTertiary: onOf(primary),
        tertiaryContainer: primaryContainer,
        onTertiaryContainer: onPrimaryContainer,
    );
}

void _ensureThemes(int accent, bool predictiveBack,
    WallpaperTextColor textMode) {
    if (_cachedAccent == accent &&
        _cachedPredictiveBack == predictiveBack &&
        _cachedTextMode == textMode &&
        _lightTheme != null) {
        return;
    }
    _cachedAccent = accent;
    _cachedPredictiveBack = predictiveBack;
    _cachedTextMode = textMode;
    final seed = Color(accent);
    PageTransitionsTheme transitions() => PageTransitionsTheme(
        builders: {
            TargetPlatform.android: predictiveBack
                ? const PredictiveBackPageTransitionsBuilder(
                    fallbackColor: Colors.transparent,
                )
                : const FadeForwardsPageTransitionsBuilder(
                    backgroundColor: Colors.transparent,
                ),
            TargetPlatform.iOS: const CupertinoPageTransitionsBuilder(),
        },
    );
}

```

```

final lightTransitions = transitions();
final darkTransitions = transitions();
final lightScheme =
    _schemeWithExactAccent(accent: seed, brightness: Brightness.light);
lightBaseScheme = lightScheme;
final lightBase = ThemeData(
    colorScheme: lightScheme,
    scaffoldBackgroundColor: Colors.transparent,
    appBarTheme: const AppBarTheme(
        backgroundColor: Color(0xFFFF4F4F6),
        elevation: 0,
        scrolledUnderElevation: 0,
        surfaceTintColor: Colors.transparent,
    ),
    cardTheme: const CardThemeData(
        color: Color(0xFFFFFFFF),
        surfaceTintColor: Colors.transparent,
    ),

    dialogTheme: DialogThemeData(
        backgroundColor: const Color(0xFFFFFFFF),
        surfaceTintColor: Colors.transparent,
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(24)),
    ),
    snackBarTheme: _toastTheme,
    pageTransitionsTheme: lightTransitions,
    useMaterial3: true,
);
lightBaseTextTheme = lightBase.textTheme;
_lightTheme = _applyWallpaperTextMode(lightBase, textMode);
final darkScheme =
    _schemeWithExactAccent(accent: seed, brightness: Brightness.dark);
darkBaseScheme = darkScheme;
_darkTheme = ThemeData(
    colorScheme: darkScheme.copyWith(
        surface: const Color(0xFF262626),
        surfaceContainerLowest: const Color(0xFF1f1f1f),
        surfaceContainerLow: const Color(0xFF262626),
        surfaceContainer: const Color(0xFF2c2c2c),
        surfaceContainerHigh: const Color(0xFF333333),
        surfaceContainerHighest: const Color(0xFF3a3a3a),
    ),
    scaffoldBackgroundColor: Colors.transparent,
    appBarTheme: const AppBarTheme(
        backgroundColor: Color(0xFF222222),
        elevation: 0,
        scrolledUnderElevation: 0,
        surfaceTintColor: Colors.transparent,
    ),
    cardTheme: const CardThemeData(

```

```

        color: Color(0xFF303030),
        surfaceTintColor: Colors.transparent,
    ),
    dialogTheme: DialogThemeData(
        backgroundColor: const Color(0xFF333333),
        surfaceTintColor: Colors.transparent,
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(24)),
    ),
    snackBarTheme: _toastTheme,
    pageTransitionsTheme: darkTransitions,
    useMaterial3: true,
);
darkBaseTextTheme = _darkTheme!.textTheme;
_darkTheme = _applyWallpaperTextMode(_darkTheme!, textMode);
}

ThemeData _applyWallpaperTextMode(ThemeData base, WallpaperTextColor mode) {
    if (mode == WallpaperTextColor.follow) return base;
    final Color onSurface;
    final Color onSurfaceVariant;
    if (mode == WallpaperTextColor.light) {
        onSurface = const Color(0xFFFFFFFF);
        onSurfaceVariant = const Color(0xB3FFFFFF);
    } else {
        onSurface = const Color(0xE6000000);
        onSurfaceVariant = const Color(0x8A000000);
    }
    final ColorScheme containers;
    if (mode == WallpaperTextColor.light) {
        containers = base.colorScheme.copyWith(
            surfaceContainerLowest: const Color(0xFF1f1f1f),
            surfaceContainerLow: const Color(0xFF262626),
            surfaceContainer: const Color(0xFF2c2c2c),
            surfaceContainerHigh: const Color(0xFF333333),
            surfaceContainerHighest: const Color(0xFF3a3a3a),
        );
    } else {
        containers = base.colorScheme.copyWith(
            surfaceContainerLowest: const Color(0xFFFFFFFF),
            surfaceContainerLow: const Color(0xFFFF7F7F8),
            surfaceContainer: const Color(0xFFE7E7F1),
            surfaceContainerHigh: const Color(0xFFE7E7EA),
            surfaceContainerHighest: const Color(0xFFD7D7E4),
        );
    }
    return base.copyWith(
        colorScheme:
            containers.copyWith(onSurface: onSurface, onSurfaceVariant: onSurfaceVariant),
        textTheme:
            base.textTheme.apply(bodyColor: onSurface, displayColor: onSurface),
    );
}

```

```

        iconTheme: IconThemeData(color: onSurface),
    );
}

@override
Widget build(BuildContext context) {
    final init = ref.watch(rustInitProvider);
    final settings = ref.watch(settingsProvider).valueOrNull;
    final accent = settings?.accentColor ?? 0xFFEC4141;
    final themeMode = switch (settings?.themeMode ?? ThemeModePreference.system) {
        ThemeModePreference.light => ThemeMode.light,
        ThemeModePreference.dark => ThemeMode.dark,
        ThemeModePreference.system => ThemeMode.system,
    };
    final cbActive = settings?.customBackground.active == true;
    final textMode = cbActive
        ? (settings!.customBackground.textMode)
        : WallpaperTextColor.follow;
    _ensureThemes(accent, settings?.enablePredictiveBack ?? false, textMode);
    final ThemeData theme = _lightTheme!;
    final ThemeData darkTheme = _darkTheme!;
    final cb = settings?.customBackground;
    if (cb?.active == true) {
        final bgPath = cb!.imagePath;
        if (bgPath.isNotEmpty) {
            WidgetsBinding.instance.addPostFrameCallback((_) {
                precacheImage(FileImage(File(bgPath)), this.context);
            });
        }
    }
    final language = settings?.language ?? AppLanguage.system;
    final locale = _localeFor(language);
    l18n.setMode(_l18nModeFor(language));
    final l10nDelegates = [
        AppLocalizations.delegate,
        GlobalMaterialLocalizations.delegate,
        GlobalWidgetsLocalizations.delegate,
        GlobalCupertinoLocalizations.delegate,
    ];

    if (init.hasValue && !_loggedHomeFirstFrame) {
        _loggedHomeFirstFrame = true;
        WidgetsBinding.instance.addPostFrameCallback((_) {
            unawaited(_runStartupAfterConsent(ref));
        });
    }

    return init.hasError
        ? MaterialApp(
            title: '$${tr('弦予音乐')}${kDebugMode ? ' • 测试' : ''}',

```

```

        debugShowCheckedModeBanner: false,
        theme: theme,
        darkTheme: darkTheme,
        themeMode: themeMode,
        locale: locale,
        localizationsDelegates: l10nDelegates,
        supportedLocales: AppLocalizations.supportedLocales,
        home: _InitErrorScreen(
            error: init.error!,
            onRetry: () => ref.invalidate(rustInitProvider),
        ),
    ),
)

: MaterialApp.router(
    key: ValueKey('app-${l18n.mode.name}'),
    title: '${tr('弦予音乐')}${kDebugMode ? ' • 测试' : ''}',
    debugShowCheckedModeBanner: false,
    theme: theme,
    darkTheme: darkTheme,
    themeMode: themeMode,
    locale: locale,
    localizationsDelegates: l10nDelegates,
    supportedLocales: AppLocalizations.supportedLocales,
    routerConfig: appRouter,
    builder: (context, child) {
        WidgetsBinding.instance.addPostFrameCallback((_) {
            final overlay = appNavigatorKey.currentState?.overlay;
            if (overlay != null) FlyingCover.instance.attach(overlay);
        });
        final fontSize = settings?.fontSize ?? AppFontSize.system;
        final textScaler = fontSize.followsSystem
            ? MediaQuery.textScalerOf(context)
            : TextScaler.linear(fontSize.scale);
        return MediaQuery(
            data: MediaQuery.of(context).copyWith(textScaler: textScaler),
            child: Stack(
                fit: StackFit.expand,
                children: [
                    ColoredBox(
                        color: appSurfaceBg(context),
                        child: const CustomBackgroundLayer(),
                    ),
                    ScrollOffsetCapture(child: child!),
                ],
            ),
        );
    },
);
}

```

```

Locale? _localeFor(AppLanguage lang) => switch (lang) {

```

```

    AppLanguage.system => null,
    AppLanguage.zhCN => const Locale('zh'),
    AppLanguage.zhTW => const Locale('zh', 'TW'),
    AppLanguage.en => const Locale('en'),
  };

  I18nMode _i18nModeFor(AppLanguage lang) => switch (lang) {
    AppLanguage.zhCN => I18nMode.zhCn,
    AppLanguage.zhTW => I18nMode.zhTw,
    AppLanguage.en => I18nMode.en,
    AppLanguage.system => _modeForSystemLocale(),
  };

  I18nMode _modeForSystemLocale() {
    final locales = WidgetsBinding.instance.platformDispatcher.locales;
    if (locales.isEmpty) return I18nMode.zhCn;
    final first = locales.first;
    switch (first.languageCode) {
      case 'en':
        return I18nMode.en;
      case 'zh':

        final cc = first.countryCode;
        if (first.scriptCode == 'Hant' ||
            cc == 'TW' ||
            cc == 'HK' ||
            cc == 'MO') {
          return I18nMode.zhTw;
        }
        return I18nMode.zhCn;
    }
    return I18nMode.zhCn;
  }
}

class _InitErrorScreen extends StatelessWidget {
  const _InitErrorScreen({required this.error, required this.onRetry});
  final Object error;
  final VoidCallback onRetry;
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: Padding(
          padding: const EdgeInsets.all(24),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
              const Icon(Icons.error_outline, size: 48),
              const SizedBox(height: 16),
            ],
          ),
        ),
      ),
    );
  }
}

```

```

        Text(tr('核心初始化失败'),
            style: const TextStyle(fontWeight: FontWeight.w600)),
        const SizedBox(height: 8),
        Text('$error', textAlign: TextAlign.center),
        const SizedBox(height: 16),
        FilledButton.icon(
            onPressed: onRetry,
            icon: const Icon(Icons.refresh),
            label: Text(tr('重试')),
        ),
    ],
),
),
),
);
}
}

```

```
// ===== lib/src/player/audio_head_cache.dart =====
```

```

import 'dart:async';
import 'dart:io';
import 'dart:typed_data';

```

```
import 'media_url.dart';
```

```

class AudioHeadEntry {
  const AudioHeadEntry({
    required this.bytes,
    required this.rangeOk,
    required this.totalLength,
    this.contentType,
    required this.storedAt,
  });

```

```
  final Uint8List bytes;
```

```
  final bool rangeOk;
```

```
  final int totalLength;
```

```
  final String? contentType;
```

```
  final DateTime storedAt;
```

```
}
```

```
class AudioHeadCache {
```

```
  AudioHeadCache._();
```

```
  static final AudioHeadCache instance = AudioHeadCache._();
```

```
static const int _maxEntries = 12;
static const int _maxTotalBytes = 24 * 1024 * 1024;
static const Duration _ttl = Duration(minutes: 15);

static const int _maxReadGuard = 8 * 1024 * 1024;

final Map<String, AudioHeadEntry> _heads = {};
int _totalBytes = 0;

final Map<String, Map<String, String>> _headersBy = {};

final Set<String> _inflight = {};

void registerHeaders(String url, Map<String, String>? headers) {
  final key = sanitizeMediaUrl(url);
  if (key.isEmpty || headers == null || headers.isEmpty) return;
  final cleaned = <String, String>{};
  headers.forEach((k, v) {
    final name = k.trim();
    final value = v.trim();
    if (name.isEmpty || value.isEmpty) return;
    if (name.toLowerCase() == 'range') return;
    cleaned[name] = value;
  });
  if (cleaned.isNotEmpty) _headersBy[key] = cleaned;
}

Map<String, String>? headersFor(String url) {
  final key = sanitizeMediaUrl(url);
  final hit = _headersBy[key];
  if (hit == null) return null;
  return Map<String, String>.of(hit);
}

AudioHeadEntry? lookup(String url) {
  final key = sanitizeMediaUrl(url);
  final entry = _heads.remove(key);
  if (entry == null) return null;
  if (DateTime.now().difference(entry.storedAt) > _ttl) {
    _totalBytes -= entry.bytes.length;
    return null;
  }
  _heads[key] = entry;
  return entry;
}

AudioHeadEntry? lookupForPlay(String url) {
  final entry = lookup(url);
  if (entry == null ||
    !entry.rangeOk ||
```

```

        entry.bytes.isEmpty ||
        entry.totalLength <= 0) {
            return null;
        }
        return entry;
    }
}

```

```

Future<bool> prefetch({
    required String url,
    Map<String, String>? headers,
    required int maxBytes,
}) async {
    final key = sanitizeMediaUrl(url);
    if (key.isEmpty || !key.startsWith('http')) return false;
    registerHeaders(key, headers);
    if (lookup(key) != null) return false;
    if (!_inflight.add(key)) return false;
    try {
        final entry = await _fetchHead(key, headers, maxBytes);
        if (entry == null) return false;
        final old = _heads.remove(key);
        if (old != null) _totalBytes -= old.bytes.length;
        _heads[key] = entry;
        _totalBytes += entry.bytes.length;
        _evict();
        return true;
    } catch (_) {
        return false;
    } finally {
        _inflight.remove(key);
    }
}

```

```

void _evict() {
    while (_heads.isNotEmpty &&
        (_heads.length > _maxEntries || _totalBytes > _maxTotalBytes)) {
        final oldestKey = _heads.keys.first;
        final entry = _heads.remove(oldestKey);
        if (entry != null) _totalBytes -= entry.bytes.length;
    }
}

```

```

Future<AudioHeadEntry?> _fetchHead(
    String url,
    Map<String, String>? headers,
    int maxBytes,
) async {
    final clamped = maxBytes.clamp(64 * 1024, _maxReadGuard);
    final client = HttpClient();
    client.connectionTimeout = const Duration(seconds: 10);

```

```

try {
    final req = await client.getUrl(Uri.parse(url));
    req.headers.set(HttpHeaders.rangeHeader, 'bytes=0-${clamped - 1}');
    headers?.forEach((k, v) {
        final name = k.trim();
        final value = v.trim();
        if (name.isEmpty || value.isEmpty) return;
        final lower = name.toLowerCase();
        if (lower == 'range' || lower == 'host' || lower == 'content-length') {
            return;
        }
        try {
            req.headers.set(name, value);
        } catch (_) {}
    });
    final resp = await req.close().timeout(const Duration(seconds: 20));

    final contentType =
        resp.headers.value(HttpHeaders.contentTypeHeader)?.toLowerCase() ??
        '';
    final nonAudio = contentType.contains('text/html') ||
        contentType.contains('application/json') ||
        contentType.contains('text/plain') ||
        contentType.contains('xml');
    if (nonAudio) {
        await resp.drain<void>().catchError((_) {});
        return null;
    }

    if (resp.statusCode == HttpStatus.partialContent) {
        final bytes = await _readUpTo(resp, clamped);
        if (bytes.isEmpty) return null;
        return AudioHeadEntry(
            bytes: bytes,
            rangeOk: true,
            totalLength: _parseTotalLength(resp),
            contentType: contentType.isNotEmpty ? contentType : null,
            storedAt: DateTime.now(),
        );
    }

    if (resp.statusCode == HttpStatus.ok) {
        await _readUpTo(resp, 64 * 1024);
        return null;
    }

    await resp.drain<void>().catchError((_) {});
    return null;
} finally {
    client.close(force: true);
}

```

```

    }
}

Future<Uint8List> _readUpTo(HttpClientResponse resp, int maxBytes) async {
  final builder = BytesBuilder(copy: false);
  await for (final chunk in resp) {
    builder.add(chunk);
    if (builder.length >= maxBytes) break;
  }
  var bytes = builder.takeBytes();
  if (bytes.length > maxBytes) {
    bytes = Uint8List.sublistView(bytes, 0, maxBytes);
  }
  return bytes;
}

int _parseTotalLength(HttpClientResponse resp) {
  final contentRange =
    resp.headers.value(HttpHeaders.contentRangeHeader) ?? '';
  final slash = contentRange.lastIndexOf('/');
  if (slash < 0 || slash == contentRange.length - 1) return 0;
  final totalStr = contentRange.substring(slash + 1).trim();
  return int.tryParse(totalStr) ?? 0;
}

void clear() {
  _heads.clear();
  _totalBytes = 0;
  _headersBy.clear();
  _inflight.clear();
}
}

// ===== lib/src/player/audio_proxy_server.dart =====

import 'dart:async';
import 'dart:convert';
import 'dart:io';
import 'dart:math';
import 'dart:typed_data';

import '../rust/api.dart' as frb;
import '../core/application_logger.dart';
import 'audio_head_cache.dart';

void probeLog(String msg) {
  AppLog.warn('proxyprobe', msg);
}

class _ByteRange {
  const _ByteRange(this.start, this.end);

```

```
    final int start;
    final int? end;
}

class _CacheStatus {
    const _CacheStatus({
        required this.exists,
        required this.complete,
        required this.failed,
        required this.total,
        required this.downloaded,
    });
    final bool exists;
    final bool complete;
    final bool failed;
    final int? total;
    final int downloaded;
}

class AudioProxyServer {
    AudioProxyServer._();

    static final AudioProxyServer instance = AudioProxyServer._();

    static const int _cacheChunk = 1 << 20;

    static const Duration _upstreamStall = Duration(seconds: 10);

    HttpServer? _server;
    String _token = '';
    Future<void>? _starting;

    bool get running => _server != null;

    Future<void> ensureStarted() async {
        if (_server != null) return;
        if (_starting != null) {
            await _starting;
            return;
        }
        final f = _start();
        _starting = f;
        try {
            await f;
        } finally {
            _starting = null;
        }
    }

    Future<void> _start() async {
```

```

    try {
        final server = await HttpServer.bind(
            InternetAddress.loopbackIPv4,
            0,
        );
        final rnd = Random.secure();
        _token = List.generate(16, (_) => rnd.nextInt(16).toRadixString(16))
            .join();
        _server = server;
        server.listen(
            (req) {
                unawaited(_onRequest(req));
            },
            onError: (_) {},
            cancelOnError: false,
        );
    } catch (_) {
        _server = null;
    }
}

```

```

String playUrlFor(String url) {
    final server = _server;
    if (server == null || !running) return url;
    if (!url.startsWith('http')) return url;

    return proxyUrlFor(url) ?? url;
}

```

```

String? proxyUrlFor(String url) {
    final server = _server;
    if (server == null || !running) return null;
    return 'http://127.0.0.1:${server.port}/${_token}/audio'
        '?u=${Uri.encodeComponent(url)}';
}

```

```

String? mvProxyUrlFor(String url, Map<String, String>? headers) {
    if (!url.startsWith('http')) return null;
    AudioHeadCache.instance.registerHeaders(url, headers);
    return proxyUrlFor(url);
}

```

```

Future<void> _onRequest(HttpRequest req) async {
    try {
        if (req.method != 'GET' && req.method != 'HEAD') {
            return await _reject(req, HttpStatus.methodNotAllowed);
        }
        if (req.uri.path != '/$_token/audio') {
            return await _reject(req, HttpStatus.notFound);
        }
    }
}

```

```

        final target = req.uri.queryParameters['u'] ?? '';
        if (!target.startsWith('http')) {
            return await _reject(req, HttpStatus.badRequest);
        }

        final upstreamHeaders = AudioHeadCache.instance.headersFor(target);
        final rawRange = req.headers.value(HttpHeaders.rangeHeader);
        final range = _parseRange(rawRange) ?? const _ByteRange(0, null);
        final head = AudioHeadCache.instance.lookupForPlay(target);

        var cacheReady = false;
        if (req.method == 'GET') {
            cacheReady = await _warmStreamCache(target, upstreamHeaders);
        }
        if (cacheReady &&
            await _tryServeFromCache(
                req, target, upstreamHeaders, head, range, rawRange != null,
            )) {
            return;
        }

        if (head != null && range.start < head.bytes.length) {
            await _serveWithHead(req, target, upstreamHeaders, head, range);
        } else {
            await _passthrough(req, target, upstreamHeaders, range);
        }
    } catch (e) {
        probeLog('onRequest error path=${req.uri.path} err=$e');
        try {
            await req.response.close();
        } catch (_) {}
    }
}

```

```

Future<bool> _warmStreamCache(
    String target,
    Map<String, String>? upstreamHeaders,
) async {
    try {
        await frb.streamCacheBeginUrlDownload(
            url: target,
            headers: jsonEncode(upstreamHeaders ?? const <String, String>{}),
        );
        return true;
    } catch (_) {
        return false;
    }
}

```

```

Future<_CacheStatus?> _cacheStatus(String target) async {

```

```

    try {
        final raw = await frb.streamCacheUrlStatus(url: target);
        final m = jsonDecode(raw) as Map<String, dynamic>;
        final total = m['total'];
        final downloaded = m['downloaded_bytes'];
        return _CacheStatus(
            exists: m['exists'] == true,
            complete: m['complete'] == true,
            failed: m['failed'] == true,
            total: total is num ? total.toInt() : null,
            downloaded: downloaded is num ? downloaded.toInt() : 0,
        );
    } catch (_) {
        return null;
    }
}

```

```

Future<_CacheStatus?> _waitForCacheTotal(
    String target,
    _CacheStatus initial,
) async {
    var st = initial;
    final waitStart = DateTime.now();
    while (!st.complete &&
        st.total == null &&
        st.downloaded <= 0 &&
        DateTime.now().difference(waitStart) < const Duration(seconds: 3)) {
        await Future<void>.delayed(const Duration(milliseconds: 150));
        final st2 = await _cacheStatus(target);
        if (st2 == null || st2.failed) {
            return null;
        }
        st = st2;
    }
    if (!st.complete && st.total == null && st.downloaded <= 0) {
        return null;
    }
    return st;
}

```

```

Future<bool> _tryServeFromCache(
    HttpRequest req,
    String target,
    Map<String, String>? upstreamHeaders,
    AudioHeadEntry? head,
    _ByteRange range,
    bool hasRangeHeader,
) async {
    final st0 = await _cacheStatus(target);
    if (st0 == null || !st0.exists || st0.failed) {

```

```

        return false;
    }

    _CacheStatus st = st0;
    if (!st0.complete && st0.total == null && head == null) {
        final waited = await _waitForCacheTotal(target, st0);
        if (waited == null) {
            probeLog('tryCache no-total timeout → fallback');
            return false;
        }
        st = waited;
    }

    final int? total = st.complete ? st.total : (head?.totalLength ?? st.total);

    final bool noTotalStream;
    if (total == null || total <= 0) {
        if (range.start == 0 && range.end == null) {
            noTotalStream = true;
        } else {
            return false;
        }
    } else {
        noTotalStream = false;
    }

    final int safeTotal = total ?? 0;

    if (!noTotalStream && range.start >= safeTotal) {
        final res = req.response;
        res.statusCode = HttpStatus.requestedRangeNotSatisfiable;
        res.headers.set(HttpHeaders.contentRangeHeader, 'bytes */$safeTotal');
        await res.close();
        return true;
    }

    final res = req.response;
    res.bufferOutput = false;
    final contentType = head?.contentType;
    if (contentType != null && contentType.isNotEmpty) {
        res.headers.set(HttpHeaders.contentTypeHeader, contentType);
    }

    var end = noTotalStream ? -1 : (range.end ?? safeTotal - 1);
    if (!noTotalStream && end >= safeTotal) end = safeTotal - 1;
    if (noTotalStream) {

        res.statusCode = HttpStatus.ok;
        probeLog('tryCache no-total serve dl=${st.downloaded}');
    } else if (hasRangeHeader) {

```

```

        res.statusCode = HttpStatus.partialContent;
        res.headers.set(HttpHeaders.acceptRangesHeader, 'bytes');
        res.headers.set(
            HttpHeaders.contentRangeHeader,
            'bytes ${range.start}-${end}/${safeTotal}',
        );
    } else {
        res.statusCode = HttpStatus.ok;
        res.headers.set(HttpHeaders.acceptRangesHeader, 'bytes');
    }
    if (!noTotalStream) {
        res.contentLength = end - range.start + 1;
    }

    var wroteAny = false;
    var pos = range.start;
    final readTimer = Stopwatch()..start();
    try {
        while (noTotalStream || pos <= end) {
            final Uint8List chunk;
            try {
                chunk = await frb.streamCacheReadUrl(
                    url: target,
                    offset: BigInt.from(pos),
                    maxLen: _cacheChunk,
                );
            } catch (e) {
                probeLog('tryCache read-error pos=$pos total=${total ?? '-'}'
                    'firstMs=${readTimer.elapsedMilliseconds}ms err=$e');
                break;
            }
            if (chunk.isEmpty) {
                if (!wroteAny && !noTotalStream) {

                    return false;
                }

                final st2 = await _cacheStatus(target);
                if (st2 == null || st2.failed) {
                    probeLog('tryCache stalled-failed pos=$pos total=${total ?? '-'}'
                        'firstMs=${readTimer.elapsedMilliseconds}ms');
                    break;
                }
                if (st2.complete && pos >= st2.downloaded) {
                    probeLog('tryCache read-Eof pos=$pos total=${total ?? '-'}'
                        'firstMs=${readTimer.elapsedMilliseconds}ms');
                    break;
                }
            }
            await Future<void>.delayed(const Duration(milliseconds: 120));
            continue;
        }
    }

```

```

    }
    var data = chunk;
    if (!noTotalStream && pos + data.length > end + 1) {
        data = Uint8List.sublistView(data, 0, end + 1 - pos);
    }
    try {
        res.add(data);
        await res.flush();
    } catch (_) {
        probeLog('tryCache client-disconnect pos=$pos total=${total ?? '-' }');
        return true;
    }
    pos += data.length;
    wroteAny = true;
}
} catch (e) {
    probeLog('tryCache loop-error pos=$pos total=${total ?? '-' } err=$e');
}
final stEnd = await _cacheStatus(target);
probeLog('tryCache serve-end pos=$pos '
    'end=${noTotalStream ? '-' : '$end'} total=${total ?? '-' } '
    'complete=${stEnd?.complete} failed=${stEnd?.failed} '
    'dl=${stEnd?.downloaded} firstMs=${readTimer.elapsedMilliseconds}ms');
try {
    await res.close();
} catch (_) {}
return true;
}

Future<void> _reject(HttpRequest req, int status) async {
    try {
        req.response.statusCode = status;
        await req.response.close();
    } catch (_) {}
}

Future<void> _serveWithHead(
    HttpRequest req,
    String target,
    Map<String, String>? upstreamHeaders,
    AudioHeadEntry head,
    _ByteRange range,
) async {
    final res = req.response;
    final headLen = head.bytes.length;
    final total = head.totalLength;

    var end = range.end ?? total - 1;
    if (end >= total) end = total - 1;
    if (end < range.start) {

```

```

        res.statusCode = HttpStatus.requestedRangeNotSatisfiable;
        res.headers.set(
            HttpHeaders.contentRangeHeader,
            'bytes */$total',
        );
        await res.close();
        return;
    }

    res.statusCode = HttpStatus.partialContent;
    res.bufferOutput = false;
    res.headers.set(HttpHeaders.acceptRangesHeader, 'bytes');
    res.headers.set(
        HttpHeaders.contentRangeHeader,
        'bytes ${range.start}-${end}/$total',
    );
    res.headers.set(
        HttpHeaders.contentTypeHeader,
        head.contentType ?? 'application/octet-stream',
    );
    res.contentLength = end - range.start + 1;

    final headEnd = end < headLen - 1 ? end : headLen - 1;
    if (headEnd >= range.start) {
        res.add(
            Uint8List.sublistView(head.bytes, range.start, headEnd + 1));
    }

    if (end >= headLen) {
        final client = HttpClient();
        client.connectionTimeout = const Duration(seconds: 10);
        final sw = Stopwatch()..start();
        try {
            final ureq = await client.getUrl(Uri.parse(target));
            ureq.headers.set(HttpHeaders.rangeHeader, 'bytes=$headLen-$end');
            _applyUpstreamHeaders(ureq, upstreamHeaders);
            final uresp = await ureq.close().timeout(const Duration(seconds: 20));
            unawaited(res.done.whenComplete(() {
                try {
                    ureq.abort();
                } catch (_) {}
            }));
        }

        final upstreamFullBody = uresp.statusCode != HttpStatus.partialContent;
        var skip = 0;
        if (upstreamFullBody) {
            skip = headEnd >= range.start ? headEnd + 1 : range.start;
        }
        var served = 0;
        var stalled = false;
        try {

```

```

        await for (final chunk
            in uresp.timeout(_upstreamStall, onTimeout: (sink) {
                stalled = true;
                sink.addError(TimeoutException('tail stall'));
            }) {
            var data = chunk;
            if (skip > 0) {
                if (data.length <= skip) {
                    skip -= data.length;
                    continue;
                }
                data = Uint8List.sublistView(
                    data is Uint8List ? data : Uint8List.fromList(data), skip);
                skip = 0;
            }
            res.add(data);
            served += data.length;
        }
    } catch (e) {
        probeLog('tail stream-error status=${uresp.statusCode} '
            'served=${served}B t=${sw.elapsedMilliseconds}ms err=${e}');
    }
    probeLog('tail end status=${uresp.statusCode} '
        'served=${served}B stalled=${stalled} t=${sw.elapsedMilliseconds}ms');
} catch (e) {
    probeLog('tail upstream-error err=${e}');
} finally {
    client.close(force: true);
}
}

try {
    await res.close();
} catch (_) {}
}

Future<void> _passthrough(
    HttpRequest req,
    String target,
    Map<String, String>? upstreamHeaders,
    _ByteRange range,
) async {
    final client = HttpClient();
    client.connectionTimeout = const Duration(seconds: 10);
    HttpClientRequest? ureq;
    try {
        ureq = await client.getUrl(Uri.parse(target));
        if (range.start > 0 || range.end != null) {
            final endStr = range.end?.toString() ?? '';
            ureq.headers.set(

```

```

        HttpHeaders.rangeHeader,
        'bytes=${range.start}-${endStr}',
    );
}
_applyUpstreamHeaders(ureq, upstreamHeaders);
final uresp = await ureq.close().timeout(const Duration(seconds: 20));

final res = req.response;
res.statusCode = uresp.statusCode;
res.bufferOutput = false;
_copyHeader(uresp, res, HttpHeaders.contentTypeHeader);
_copyHeader(uresp, res, HttpHeaders.contentLengthHeader);
_copyHeader(uresp, res, HttpHeaders.contentRangeHeader);
_copyHeader(uresp, res, HttpHeaders.acceptRangesHeader);

final sw = Stopwatch()..start();
var served = 0;
var stalled = false;
unawaited(res.done.whenComplete(() {
    try {
        ureq?.abort();
    } catch (_) {}
}));
try {
    await for (final chunk
        in uresp.timeout(_upstreamStall, onTimeout: (sink) {
            stalled = true;
            sink.addError(TimeoutException('passthrough stall'));
        }) {
        res.add(chunk);
        served += chunk.length;
    }
} catch (e) {
    probeLog('passthrough stream-error status=${uresp.statusCode} '
        'served=${served}B t=${sw.elapsedMilliseconds}ms err=$e');
}

probeLog('passthrough end status=${uresp.statusCode} '
    'cl=${uresp.headers.value(HttpHeaders.contentLengthHeader) ?? '-' } '
    'served=${served}B stalled=$stalled t=${sw.elapsedMilliseconds}ms');
try {
    await res.close();
} catch (_) {}
} catch (e) {
    probeLog('passthrough upstream-error err=$e');
    try {
        req.response.statusCode = HttpStatus.badGateway;
        await req.response.close();
    } catch (_) {}
} finally {

```

```

        client.close(force: true);
    }
}

void _applyUpstreamHeaders(
    HttpClientRequest ureq,
    Map<String, String>? headers,
) {
    headers?.forEach((k, v) {
        final name = k.trim();
        final value = v.trim();
        if (name.isEmpty || value.isEmpty) return;
        final lower = name.toLowerCase();
        if (lower == 'host' ||
            lower == 'content-length' ||
            lower == 'range') {
            return;
        }
        try {
            ureq.headers.set(name, value);
        } catch (_) {}
    });
}

void _copyHeader(
    HttpClientResponse from,
    HttpResponse to,
    String name,
) {
    final v = from.headers.value(name);
    if (v == null || v.isEmpty) return;
    try {
        to.headers.set(name, v);
    } catch (_) {}
}

_ByteRange? _parseRange(String? raw) {
    if (raw == null) return null;
    final m = RegExp(r'^bytes=(\d+)-(\d*)$').firstMatch(raw.trim());
    if (m == null) return null;
    final start = int.tryParse(m.group(1)!);
    if (start == null) return null;
    final endStr = m.group(2);
    final end = endStr == null || endStr.isEmpty ? null : int.tryParse(endStr);
    if (end != null && end < start) return null;
    return _ByteRange(start, end);
}

// ===== lib/src/player/cast_provider.dart =====
import 'dart:async';

```

```
import 'dart:convert';
import 'dart:math';

import 'package:flutter/services.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:shared_preferences/shared_preferences.dart';

import '../core/application_logger.dart';
import '../core/settings.dart';
import '../rust/api.dart';
import '../widgets/app_toast.dart';
import '../navigation/routes.dart';
import '../il8n/il8n.dart';
import 'player_provider.dart';

class CastState {
  final String phase;

  final String? deviceJson;
  final String deviceName;
  final String deviceModel;
  final double tvPosition;
  final double tvDuration;

  final String tvState;

  final bool rendererRunning;
  final int rendererPort;
  final String rendererName;

  const CastState({
    this.phase = 'idle',
    this.deviceJson,
    this.deviceName = '',
    this.deviceModel = '',
    this.tvPosition = 0,
    this.tvDuration = 0,
    this.tvState = 'STOPPED',
    this.rendererRunning = false,
    this.rendererPort = 0,
    this.rendererName = '',
  });

  bool get isCasting => phase == 'casting' && deviceJson != null;

  CastState dropCast() => CastState(
    rendererRunning: rendererRunning,
    rendererPort: rendererPort,
    rendererName: rendererName,
  );
}
```

```
CastState copyWith({
    String? phase,
    String? deviceJson,
    String? deviceName,
    String? deviceModel,
    double? tvPosition,
    double? tvDuration,
    String? tvState,
    bool? rendererRunning,
    int? rendererPort,
    String? rendererName,
}) {
    return CastState(
        phase: phase ?? this.phase,
        deviceJson: deviceJson ?? this.deviceJson,
        deviceName: deviceName ?? this.deviceName,
        deviceModel: deviceModel ?? this.deviceModel,
        tvPosition: tvPosition ?? this.tvPosition,
        tvDuration: tvDuration ?? this.tvDuration,
        tvState: tvState ?? this.tvState,
        rendererRunning: rendererRunning ?? this.rendererRunning,
        rendererPort: rendererPort ?? this.rendererPort,
        rendererName: rendererName ?? this.rendererName,
    );
}

const int _ttlRefreshMs = 510000;

const MethodChannel _dlnaChannel = MethodChannel('xianyu/dlna');

const String _kRendererUdn = 'dlna.renderer.udn';

class CastNotifier extends StateNotifier<CastState> {
    CastNotifier(this._ref) : super(const CastState());

    final Ref _ref;

    Timer? _pollTimer;
    Timer? _reportTimer;
    bool _dmrLoopRunning = false;
    int _consecutiveErrors = 0;
    int _lastVolumeSent = -1;

    bool _batteryPromptShown = false;

    String _mediaToken = '';
    String _mediaUrl = '';
    Map<String, String> _mediaHeaders = {};
```

..... (中间部分略)

```

        for row in rows.flatten() {
            let offset = row.0;
            if offset >= 0 && offset < 7 {
                recent_activity[offset as usize] = row.1;
            }
        }
    }
}

Ok(BehaviorStats {
    total_plays,
    total_duration,
    top_songs,
    top_songs_by_duration,
    top_artists,
    top_albums,
    hour_distribution,
    recent_activity,
}))
}

#[derive(Serialize)]
pub struct ListenDurations {

    pub daily: i64,

    pub weekly: i64,

    pub total: i64,

    pub today_play_count: i64,
}

pub fn get_listen_durations(conn: &rusqlite::Connection) -> Result<ListenDurations, String> {

    let daily_ms: i64 = conn
        .query_row(
            "SELECT COALESCE(play_time_ms, 0) FROM daily_stats
            WHERE date = strftime('%Y-%m-%d', 'now', 'localtime')",
            [],
            |row| row.get(0),
        )
        .unwrap_or(0);

    let weekly_ms: i64 = conn
        .query_row(
            "SELECT COALESCE(SUM(play_time_ms), 0) FROM daily_stats
            WHERE date >= strftime('%Y-%m-%d', 'now', 'localtime', '-6 days')",
            [],
            |row| row.get(0),
        )

```

```

        .unwrap_or(0);

let total_ms: i64 = conn
    .query_row(
        "SELECT COALESCE(total_play_time_ms, 0) FROM global_stats WHERE id = 1",
        [],
        |row| row.get(0),
    )
    .unwrap_or(0);

let today_count: i64 = conn
    .query_row(
        "SELECT COALESCE(play_count, 0) FROM daily_stats
        WHERE date = strftime('%Y-%m-%d', 'now', 'localtime')",
        [],
        |row| row.get(0),
    )
    .unwrap_or(0);

Ok(ListenDurations {
    daily: daily_ms / 1000,
    weekly: weekly_ms / 1000,
    total: total_ms / 1000,
    today_play_count: today_count,
})
}

#[cfg(test)]
mod playback_count_tests {
    use super::*;

    #[test]
    fn periodic_time_flushes_only_increment_play_count_once() {
        let conn = rusqlite::Connection::open_in_memory().expect("open in-memory database");
        crate::database::schema::ensure_base_schema(&conn).expect("create schema");
        let identity = PortableSongIdentity {
            title: "Demo".to_string(),
            artist: "Artist".to_string(),
            album: "Album".to_string(),
            duration_ms: 195_000,
            track_number: Some(1),
        };

        record_aggregate_play(&conn, &identity, 1_700_000_000, 30_000, false, false, true)
            .expect("record initial play chunk");
        for offset in 1..=12 {
            record_aggregate_play(
                &conn,
                &identity,
                1_700_000_000 + offset,
            )
        }
    }
}

```

```

        30_000,
        false,
        false,
        false,
    )
    .expect("record time-only chunk");
}

let (play_count, play_time_ms): (i64, i64) = conn
    .query_row(
        "SELECT total_play_count, total_play_time_ms FROM global_stats WHERE id = 1",
        [],
        |row| Ok((row.get(0)?, row.get(1)?)),
    )
    .expect("read global stats");
assert_eq!(play_count, 1);
assert_eq!(play_time_ms, 390_000);

let recent_count: i64 = conn
    .query_row("SELECT COUNT(*) FROM recent_plays", [], |row| row.get(0))
    .expect("read recent plays");
assert_eq!(recent_count, 1);
}
}

// ===== rust/src/toolbox.rs =====

use crate::music::tags::{
    extract_text_metadata, read_tagged_file_from_path, write_metadata_to_file, EmbedMetadataRequest,
};
use crate::music::utils::is_supported_library_extension;
use crate::security::path_validator;
use lofty::prelude::*;
use regex::Regex;
use serde::{Deserialize, Serialize};
use std::fs;
use std::path::{Path, PathBuf};
use std::sync::{Arc, Mutex, OnceLock};
use walkdir::WalkDir;

static TRACK_PREFIX_RE: OnceLock<Regex> = OnceLock::new();
static SOURCE_PREFIX_RE: OnceLock<Regex> = OnceLock::new();

#[derive(Debug, Serialize, Deserialize)]
pub struct RenameConfig {
    pub mode: String,
    pub template: String,
    pub remove_track_prefix: bool,
    pub remove_source_prefix: bool,
}

```

```

#[derive(Debug, Serialize, Deserialize)]
pub struct RenamePreview {
    pub original_path: String,
    pub original_name: String,
    pub new_name: String,
    pub status: String,
    pub error: Option<String>,
}

#[derive(Debug, Serialize, Deserialize)]
pub struct RenameOperation {
    pub original_path: String,
    pub new_name: String,
}

fn sanitize_filename(name: &str) -> String {
    let invalid_chars = ['<', '>', ':', '"', '/', '\\', '|', '?', '*'];
    let mut sanitized = String::new();
    for c in name.chars() {
        if invalid_chars.contains(&c) {
            sanitized.push('_');
        } else {
            sanitized.push(c);
        }
    }
    sanitized.trim().to_string()
}

fn process_file(path: &Path, config: &RenameConfig) -> RenamePreview {
    let original_name = path
        .file_name()
        .unwrap_or_default()
        .to_string_lossy()
        .to_string();
    let original_path_str = path.to_string_lossy().to_string();
    let ext = path
        .extension()
        .unwrap_or_default()
        .to_string_lossy()
        .to_string();

    if config.mode == "tags" || config.mode == "auto" {
        if let Ok(tagged_file) = read_tagged_file_from_path(path) {
            let metadata = extract_text_metadata(&tagged_file);
            let title = metadata.title.unwrap_or_default();
            let artist = metadata.artist.unwrap_or_default();
            let album = metadata.album.unwrap_or_default();

            let year = tagged_file
                .primary_tag()

```

```

        .and_then(|tag| tag.year())
        .map(|y| y.to_string())
        .unwrap_or_default();

let track = tagged_file
    .primary_tag()
    .and_then(|tag| tag.track())
    .map(|t| format!("{:02}", t))
    .unwrap_or_default();

if !title.is_empty() {
    let mut new_name_base = config.template.clone();
    new_name_base = new_name_base.replace("{title}", &title);
    new_name_base = new_name_base.replace("{artist}", &artist);
    new_name_base = new_name_base.replace("{album}", &album);
    new_name_base = new_name_base.replace("{year}", &year);
    new_name_base = new_name_base.replace("{track}", &track);

    let new_name = format!("{}", sanitize_filename(&new_name_base), ext);

    if new_name != original_name {
        return RenamePreview {
            original_path: original_path_str,
            original_name,
            new_name,
            status: "tags".to_string(),
            error: None,
        };
    } else if config.mode == "tags" {
        return RenamePreview {
            original_path: original_path_str,
            original_name: original_name.clone(),
            new_name: original_name,
            status: "skipped".to_string(),
            error: Some("Already named correctly".to_string()),
        };
    }
}

if config.mode == "tags" {
    return RenamePreview {
        original_path: original_path_str,
        original_name: original_name.clone(),
        new_name: original_name,
        status: "skipped".to_string(),
        error: Some("Missing tags".to_string()),
    };
}
}

```

```

    if config.mode == "rules" || config.mode == "auto" {
        let mut cleaned_name = original_name.clone();

        if let Some(stem) = path.file_stem() {
            let mut stem_str = stem.to_string_lossy().to_string();

            if config.remove_track_prefix {
                let re = TRACK_PREFIX_RE.get_or_init(|| Regex::new(r"^\d+[\.\-\s]+").unwrap());
                stem_str = re.replace(&stem_str, "").to_string();
            }

            if config.remove_source_prefix {
                let re = SOURCE_PREFIX_RE.get_or_init(|| Regex::new(r"^\s*[.\*?\\]\s*").unwrap());
                stem_str = re.replace(&stem_str, "").to_string();
            }

            cleaned_name = format!("{}", stem_str.trim(), ext);
        }

        if cleaned_name != original_name {
            return RenamePreview {
                original_path: original_path_str,
                original_name,
                new_name: cleaned_name,
                status: "rules".to_string(),
                error: None,
            };
        }
    }

    RenamePreview {
        original_path: original_path_str,
        original_name: original_name.clone(),
        new_name: original_name,
        status: "skipped".to_string(),
        error: Some("No rules matched or missing tags".to_string()),
    }
}

pub fn preview_rename(root_path: String, config: RenameConfig) -> Result<Vec<RenamePreview>, String>
{
    let validated_root = path_validator::validate_path(&root_path, None)?;
    let root_path = validated_root.to_string_lossy().to_string();
    let mut results = Vec::new();

    for entry in WalkDir::new(root_path)
        .max_depth(1)
        .into_iter()
        .filter_map(|e| e.ok())
    {
        let path = entry.path();

```

```

        if path.is_file() {
            if let Some(ext) = path.extension() {
                let ext = ext.to_string_lossy().to_lowercase();
                if is_supported_library_extension(&ext) {
                    results.push(process_file(path, &config));
                }
            }
        }
    }

    results.sort_by(|a, b| {
        let a_changed = a.status != "skipped";
        let b_changed = b.status != "skipped";
        if a_changed && !b_changed {
            std::cmp::Ordering::Less
        } else if !a_changed && b_changed {
            std::cmp::Ordering::Greater
        } else {
            a.original_name.cmp(&b.original_name)
        }
    });

    Ok(results)
}

pub fn apply_rename(operations: Vec<RenameOperation>) -> Result<u32, String> {
    let mut success_count = 0;

    for mut op in operations {
        let validated_path = path_validator::validate_path(&op.original_path, None)?;
        op.original_path = validated_path.to_string_lossy().to_string();
        op.new_name = path_validator::sanitize_filename_component(&op.new_name)?;
        let src = PathBuf::from(&op.original_path);
        if let Some(parent) = src.parent() {
            let dest = parent.join(&op.new_name);
            if fs::rename(&src, &dest).is_ok() {
                success_count += 1;
            }
        }
    }

    Ok(success_count)
}

pub fn refresh_folder_songs(
    conn: Arc<Mutex<rusqlite::Connection>>,
    folder_path: String,
    minimum_duration_seconds: Option<u32>,
    cover_cache_dir: Option<PathBuf>,
) -> Result<Vec<crate::music::types::Song>, String> {

```

```

    let validated = path_validator::validate_path(&folder_path, None)?;
    let folder_path = validated.to_string_lossy().to_string();
    crate::music::scanner::scan_single_directory_internal(
        folder_path,
        conn,
        None,
        None,
        1,
        1,
        crate::music::scanner::ScanOptions::from_minimum_duration_seconds(minimum_duration_seconds),
        cover_cache_dir,
    )
}

pub fn file_exists(path: String) -> bool {
    if path_validator::validate_path(&path, None).is_err() {
        return false;
    }
    std::path::Path::new(&path).is_file()
}

pub fn resolve_download_path(
    directory: String,
    file_name: String,
    overwrite_existing: bool,
) -> Result<String, String> {
    let dir = path_validator::validate_path(&directory, None)?;
    let file_name = path_validator::sanitize_filename_component(&file_name)?;
    let direct = dir.join(&file_name);

    if overwrite_existing || !direct.exists() {
        std::fs::create_dir_all(&dir).map_err(|e| format!("创建下载目录失败: {e}"))?;
        return Ok(direct.to_string_lossy().to_string());
    }

    let dot = file_name.rfind('.');
    let (stem, ext) = match dot {
        Some(idx) => (&file_name[..idx], &file_name[idx..]),
        None => (file_name.as_str(), ""),
    };

    for i in 1..1000 {
        let candidate_name = format!("{stem} ({i}){ext}");
        let candidate = dir.join(&candidate_name);
        if !candidate.exists() {
            return Ok(candidate.to_string_lossy().to_string());
        }
    }

    Ok(direct.to_string_lossy().to_string())
}

```

```
}

```

```
fn sanitize_download_filename(name: &str) -> String {
    let sanitized: String = name
        .chars()
        .map(|c| {
            if c.is_control() || matches!(c, '<' | '>' | ':' | '"' | '/' | '\\\' | '|' | '?' | '*') {
                , ,
            } else {
                c
            }
        })
        .collect();
    let collapsed: String = sanitized.split_whitespace().collect::<Vec<_>>().join(" ");
    let trimmed = collapsed.trim();
    if trimmed.is_empty() {
        return "download".to_string();
    }
    trimmed.chars().take(180).collect()
}
```

```
fn ext_from_url(url: &str) -> String {
    let path = match request::Url::parse(url) {
        Ok(u) => u.path().to_string(),
        Err(_) => return String::new(),
    };
    let dot = match path.rfind('.') {
        Some(idx) => idx,
        None => return String::new(),
    };
    let ext = path[dot..].to_lowercase();
    match ext.as_str() {
        ".mp3" | ".flac" | ".wav" | ".m4a" | ".aac" | ".ape" | ".ogg" | ".wma" => ext,
        _ => String::new(),
    }
}
```

```
fn is_lossless_quality(quality: &str) -> bool {
    matches!(quality, "flac" | "flac24bit" | "hires" | "vinyl" | "master")
}
```

```
fn ext_from_quality(quality: &str) -> String {
    if is_lossless_quality(quality) {
        ".flac".to_string()
    } else {
        ".mp3".to_string()
    }
}
```

```
fn build_filename_base(title: &str, artist: &str, album: &str, style: &str) -> String {
```

```

    let title = if title.is_empty() {
        "未知歌曲"
    } else {
        title
    };
    let parts: Vec<&str> = match style {
        "title-artist" => vec![title, artist],
        "title-artist-album" => vec![title, artist, album],
        _ => vec![artist, title],
    };
    let joined: String = parts
        .iter()
        .map(|p| p.trim().to_string())
        .filter(|p| !p.is_empty())
        .collect::<Vec<_>>()
        .join(" - ");
    if joined.is_empty() {
        title.to_string()
    } else {
        joined
    }
}

```

```

fn build_download_filename(
    title: &str,
    artist: &str,
    album: &str,
    url: &str,
    quality: &str,
    keep_source_filename: bool,
    style: &str,
    cek: &str,
) -> String {

    let ext = if !cek.is_empty() {
        ".m4a".to_string()
    } else {
        let e = ext_from_url(url);
        if e.is_empty() {
            ext_from_quality(quality)
        } else {
            e
        }
    };

    if keep_source_filename {
        if let Ok(u) = request::Url::parse(url) {
            let path = u.path();
            if let Some(base) = path.rsplit('/').next() {
                if let Some(dot_idx) = base.rfind('.') {

```

```

        let stem = &base[..dot_idx];
        let decoded = urlencoding::decode(stem)
            .map(|cow| cow.into_owned())
            .unwrap_or_else(|_| stem.to_string());
        if !decoded.is_empty() {
            return format!("{}", sanitize_download_filename(&decoded), ext);
        }
    }
}

}

}

let base = build_filename_base(title, artist, album, style);
format!("{}", sanitize_download_filename(&base), ext)
}

```

```

#[allow(clippy::too_many_arguments)]
pub fn resolve_download_full_path(
    directory: String,
    title: String,
    artist: String,
    album: String,
    url: String,
    quality: String,
    keep_source_filename: bool,
    file_name_style: String,
    overwrite_existing: bool,
    cek: Option<String>,
) -> Result<String, String> {
    let validated_dir = path_validator::validate_path(&directory, None)?;
    let directory = validated_dir.to_string_lossy().to_string();
    let file_name = build_download_filename(
        &title,
        &artist,
        &album,
        &url,
        &quality,
        keep_source_filename,
        &file_name_style,
        cek.as_deref().unwrap_or(""),
    );
    let file_name = path_validator::sanitize_filename_component(&file_name)?;
    resolve_download_path(directory, file_name, overwrite_existing)
}

```

```

pub fn build_download_basename(
    title: String,
    artist: String,
    album: String,
    file_name_style: String,

```

```

) -> String {
    let base = build_filename_base(&title, &artist, &album, &file_name_style);
    let cleaned = sanitize_download_filename(&base);
    path_validator::sanitize_filename_component(&cleaned).unwrap_or_else(|_| "download".to_string())
}

```

```
const DOWNLOAD_HISTORY_FILE: &str = "download_history.json";
```

```

pub async fn check_update_by_rust(owner: String, repo: String) -> Result<String, String> {
    let url = format!("https://api.github.com/repos/{owner}/{repo}/releases/latest");

    let client = reqwest::Client::builder()
        .timeout(std::time::Duration::from_secs(10))
        .dns_resolver(crate::security::ssrf::pinned_dns_resolver())
        .user_agent("XY-Music-Updater")
        .build()
        .map_err(|e| format!("创建更新请求失败: {e}"))?;

    client
        .get(&url)
        .header("Accept", "application/vnd.github+json")
        .send()
        .await
        .map_err(|e| format!("请求更新接口失败: {e}"))?
        .error_for_status()
        .map_err(|e| format!("更新接口返回错误状态: {e}"))?
        .text()
        .await
        .map_err(|e| format!("读取更新数据失败: {e}"))
}

```

```
#[derive(Debug, Clone, serde::Serialize)]
```

```

pub struct SongDownloadProgress {
    pub progress: f64,
    pub downloaded: u64,
    pub total: u64,
    pub speed: f64,
}

```

```

pub async fn download_online_song(
    url: String,
    dest_path: String,
    ekey: Option<String>,
    cek: Option<String>,
    headers: Option<std::collections::HashMap<String, String>>,
) -> Result<String, String> {
    use std::time::Instant;
    use tokio::fs::File;
    use tokio::io::AsyncWriteExt;

```

```

    if !(url.starts_with("http://") || url.starts_with("https://")) {
        return Err("无效的下载链接".to_string());
    }

```

```

crate::security::ssrf::validate_outbound_url(&url)
    .await
    .map_err(|e| format!("下载链接校验失败: {e}"))?;

```

```

let client = request::Client::builder()
    .timeout(std::time::Duration::from_secs(600))

    .redirect(crate::security::ssrf::ssrf_redirect_policy())

    .dns_resolver(crate::security::ssrf::pinned_dns_resolver())
    .user_agent("Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko
) Chrome/120.0.0.0 Safari/537.36")
    .build()
    .map_err(|e| format!("创建下载请求客户端失败: {e}"))?;

```

```

let send_with_range = |with_range: bool| {
    let mut builder = client.get(&url).header(
        "Accept",
        "audio/webm, audio/ogg, audio/wav, audio/*;q=0.9, application/ogg;q=0.7, video/*;q=0.6, */*;q=
0.5",
    );
    if with_range {
        builder = builder.header("Range", "bytes=0-");
    }
    if let Some(ref hdrs) = headers {
        for (key, value) in hdrs {
            if key.eq_ignore_ascii_case("accept") || key.eq_ignore_ascii_case("range") {
                continue;
            }
            builder = builder.header(key.as_str(), value.as_str());
        }
    }
    builder.send()
};

```

```

let mut response = send_with_range(true)
    .await
    .map_err(|e| format!("发送下载请求失败: {e}"))?;
let status_code = response.status().as_u16();
if !response.status().is_success()
    && (status_code == 502 || status_code == 416 || status_code == 403)
{
    response = send_with_range(false)
        .await
        .map_err(|e| format!("发送下载请求（无 Range 回退）失败: {e}"))?;
}
if !response.status().is_success() {
    return Err(format!("下载服务器返回错误状态: {}", response.status()));
}

```

```

    }

    let total_size = response.content_length().unwrap_or(0);

    let dest = PathBuf::from(&dest_path);
    if let Some(parent) = dest.parent() {
        tokio::fs::create_dir_all(parent)
            .await
            .map_err(|e| format!("创建下载目录失败: {e}"))?;
    }

    let mut file = File::create(&dest)
        .await
        .map_err(|e| format!("创建目标文件失败: {e}"))?;
    let mut downloaded: u64 = 0;
    let start_time = Instant::now();

    let mut response = response;
    loop {
        match response.chunk().await {
            Ok(Some(chunk)) => {
                file.write_all(&chunk)
                    .await
                    .map_err(|e| format!("写入文件失败: {e}"))?;
                downloaded += chunk.len() as u64;
            }
            Ok(None) => break,
            Err(e) => {
                drop(file);
                let _ = tokio::fs::remove_file(&dest).await;
                return Err(format!("下载数据分块失败: {e}"));
            }
        }
    }

    file.flush()
        .await
        .map_err(|e| format!("刷新文件缓存失败: {e}"))?;
    drop(file);

    if total_size > 0 && downloaded < total_size {
        let _ = tokio::fs::remove_file(&dest).await;
        return Err(format!(
            "下载不完整 ({downloaded}/{total_size} 字节)，可能被其他下载器（如 IDM）拦截。请在下载器设置中排除本应用，或临时退出下载器后重试。"
        ));
    }

    if let Some(ref ek) = ekey {
        if !ek.is_empty() {
            match decrypt_qmc_file_inplace(&dest, ek) {

```

```

        Ok(_) => {}
        Err(e) => {
            return Err(format!("QMC2 解密失败: {e}"));
        }
    }
}

} else if let Some(ref ck) = cek {
    if !ck.is_empty() {

        crate::player::stream_cache::decrypt_cenc_file(&dest, ck)
            .map_err(|e| format!("CENC 解密失败: {e}"))?;
    }
} else if let Some(extracted_ekey) = try_extract_ekey_from_file(&dest) {
    match decrypt_qmc_file_inplace(&dest, &extracted_ekey) {
        Ok(_) => {}
        Err(e) => {
            return Err(format!("QMC2 解密失败 (footer ekey) : {e}"));
        }
    }
}

let _ = start_time;
Ok(dest.to_string_lossy().to_string())
}

fn try_extract_ekey_from_file(path: &Path) -> Option<String> {
    let metadata = fs::metadata(path).ok()?;
    let file_size = metadata.len();
    if file_size < 8 {
        return None;
    }

    let tail_size = (file_size.min(4096)) as usize;
    let mut file = fs::File::open(path).ok()?;
    use std::io::{Read, Seek, SeekFrom};
    file.seek(SeekFrom::Start(file_size - tail_size as u64)).ok()?;
    let mut tail = vec![0u8; tail_size];
    file.read_exact(&mut tail).ok()?;

    crate::player::qmc2::extract_ekey_from_footer(&tail)
}

fn decrypt_qmc_file_inplace(path: &Path, ekey: &str) -> Result<u64, String> {
    let crypto = crate::player::qmc2::QmcCrypto::from_ekey(ekey)
        .map_err(|e| format!("ekey 解析失败: {e}"))?;
    decrypt_with_crypto_inplace(path, &crypto)
}

fn decrypt_with_crypto_inplace(path: &Path, crypto: &crate::player::qmc2::QmcCrypto) -> Result<u64,
String> {
    use std::io::{Read, Write};

```

```

let file_size = fs::metadata(path)
    .map_err(|e| format!("读取文件元数据失败: {e}"))?
    .len();

let temp_path = path.with_extension("qmc_tmp_dec");

{
    let mut input = fs::File::open(path)
        .map_err(|e| format!("打开加密文件失败: {e}"))?;
    let mut output = fs::File::create(&temp_path)
        .map_err(|e| format!("创建临时解密文件失败: {e}"))?;

    let mut offset: u64 = 0;
    let mut buf = vec![0u8; 64 * 1024];

    loop {
        let n = input.read(&mut buf)
            .map_err(|e| format!("读取加密数据失败: {e}"))?;
        if n == 0 {
            break;
        }
        crypto.decrypt(offset as usize, &mut buf[..n]);
        output.write_all(&buf[..n])
            .map_err(|e| format!("写入解密数据失败: {e}"))?;
        offset += n as u64;
    }

    output.flush()
        .map_err(|e| format!("刷新解密文件失败: {e}"))?;
}

fs::rename(&temp_path, path)
    .map_err(|e| {
        let _ = fs::remove_file(&temp_path);
        format!("替换原文件失败: {e}")
    })?;

Ok(file_size)
}

fn read_file_header(path: &Path) -> Result<Vec<u8>, String> {
    use std::io::Read;
    let mut file = fs::File::open(path).map_err(|e| format!("打开文件失败: {e}"))?;
    let mut header = vec![0u8; 16];
    let mut filled = 0;
    while filled < header.len() {
        let n = file.read(&mut header[filled..]).map_err(|e| format!("读取文件头失败: {e}"))?;
        if n == 0 {
            break;
        }
    }
}

```

```

    }

    filled += n;
}

header.truncate(filled);
Ok(header)
}

fn is_qmc1_extension(path: &Path) -> bool {
    let Some(name) = path.file_name().and_then(|n| n.to_str()) else {
        return false;
    };
    let lower = name.to_lowercase();
    matches!(
        lower.as_str(),
        "qmcflac" | "qmcmp3" | "qmcogg" | "qmcwav" | "qmcape" | "qmcwma"
    ) || lower.starts_with("qmc")
}

fn rename_to_audio_extension(path: &Path) -> Result<Option<PathBuf>, String> {
    let header = read_file_header(path)?;
    let Some(real_ext) = crate::player::qmc2::detect_audio_extension(&header) else {
        return Ok(None);
    };
    let cur_ext = path
        .extension()
        .and_then(|e| e.to_str())
        .map(|e| e.to_lowercase())
        .unwrap_or_default();
    if cur_ext == real_ext {
        return Ok(None);
    }
    let stem = path.file_stem().unwrap_or_default();
    let new_path = path.with_file_name(format!("{}.{}", stem.to_string_lossy(), real_ext));
    if new_path.exists() && new_path != path {

        return Ok(None);
    }
    fs::rename(path, &new_path).map_err(|e| format!("修正扩展名失败: {e}"))?;
    Ok(Some(new_path))
}

pub fn decrypt_qmc_file_standalone(
    file_path: String,
    ekey: Option<String>,
) -> Result<String, String> {
    let path = path_validator::validate_path(&file_path, None)?;

    if !path.is_file() {
        return Err(format!("文件不存在: {}", path.display()));
    }
}

```

```

let actual_ekey = if let Some(ref ek) = ekey {
    if !ek.is_empty() {
        Some(ek.clone())
    } else {
        try_extract_ekey_from_file(&path)
    }
} else {
    try_extract_ekey_from_file(&path)
};

let (decrypt_result, crypto_name) = if let Some(ek) = actual_ekey {
    let crypto = crate::player::qmc2::QmcCrypto::from_ekey(&ek)
        .map_err(|e| format!("ekey 解析失败: {e}"))?;
    (decrypt_with_crypto_inplace(&path, &crypto), "QMC2")
} else if is_qmcl_extension(&path) {

    let crypto = crate::player::qmc2::QmcCrypto::qmcl();
    let backup = path.with_extension("qmc_tmp_backup");
    fs::copy(&path, &backup).map_err(|e| format!("备份原文件失败: {e}"))?;
    let result = decrypt_with_crypto_inplace(&path, &crypto);
    match result {
        Ok(_) => {

            let header = read_file_header(&path).unwrap_or_default();
            if crate::player::qmc2::detect_audio_extension(&header).is_none() {
                let _ = fs::rename(&backup, &path);
                return Err("解密结果不是有效音频（该文件可能不是 QMC1 格式）".to_string());
            }
            let _ = fs::remove_file(&backup);
            (Ok(0), "QMC1")
        }
        Err(e) => {
            let _ = fs::rename(&backup, &path);
            return Err(format!("QMC1 解密失败: {e}"));
        }
    }
} else {
    return Ok(serde_json::json!({
        "status": "not_encrypted",
        "outputPath": path.to_string_lossy(),
        "renamedTo": null,
    }));
    .to_string();
};

match decrypt_result {
    Ok(_) => {
        let renamed_to = rename_to_audio_extension(&path)?;
        let output = renamed_to.as_ref().unwrap_or(&path);
    }
}

```

```

        Ok(serde_json::json!({
            "status": "decrypted",
            "outputPath": output.to_string_lossy(),
            "renamedTo": renamed_to.as_ref().map(|p| p.to_string_lossy().to_string()),
            "crypto": crypto_name,
        }))
    }
    .to_string())
}

Err(e) => Err(format!("{crypto_name} 解密失败: {e}")),
}
}

pub fn decrypt_qmc_file(file_path: String, ekey: Option<String>) -> Result<bool, String> {
    let path = path_validator::validate_path(&file_path, None)?;

    if !path.is_file() {
        return Err(format!("文件不存在: {}", path.display()));
    }

    let actual_ekey = if let Some(ref ek) = ekey {
        if !ek.is_empty() {
            Some(ek.clone())
        } else {
            try_extract_ekey_from_file(&path)
        }
    } else {
        try_extract_ekey_from_file(&path)
    };

    if let Some(ek) = actual_ekey {
        match decrypt_qmc_file_inplace(&path, &ek) {
            Ok(_) => Ok(true),
            Err(e) => Err(format!("QMC2 解密失败: {e}")),
        }
    } else {
        Ok(false)
    }
}

pub async fn save_download_lyrics(content: String, dest_path: String) -> Result<String, String> {
    write_text_file(content, dest_path).await
}

pub async fn write_text_file(content: String, dest_path: String) -> Result<String, String> {
    let dest = path_validator::validate_path(&dest_path, None)?;
    if let Some(parent) = dest.parent() {
        tokio::fs::create_dir_all(parent)
            .await
            .map_err(|e| format!("创建目录失败: {e}"))?;
    }
}

```

```

        tokio::fs::write(&dest, content)
            .await
            .map_err(|e| format!("写入文件失败: {e}"))?;
    Ok(dest.to_string_lossy().to_string())
}

#[derive(Debug, Serialize)]
pub struct FetchedImage {
    pub data: Vec<u8>,
    pub mime: String,
}

pub async fn fetch_image_bytes(url: String) -> Result<FetchedImage, String> {
    if !(url.starts_with("http://") || url.starts_with("https://")) {
        return Err("无效的图片链接".to_string());
    }

    crate::security::ssrf::validate_outbound_url(&url)
        .await
        .map_err(|e| format!("图片链接校验失败: {e}"))?;

    let client = request::Client::builder()
        .timeout(std::time::Duration::from_secs(30))

        .redirect(crate::security::ssrf::ssrf_redirect_policy())

        .dns_resolver(crate::security::ssrf::pinned_dns_resolver())
        .user_agent("Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko
) Chrome/120.0.0.0 Safari/537.36")
        .build()
        .map_err(|e| format!("创建请求客户端失败: {e}"))?;

    let response = client
        .get(&url)
        .send()
        .await
        .map_err(|e| format!("请求图片失败: {e}"))?;

    if !response.status().is_success() {
        return Err(format!("图片服务器返回错误状态: {}", response.status()));
    }

    let mime = response
        .headers()
        .get(request::header::CONTENT_TYPE)
        .and_then(|v| v.to_str().ok())
        .unwrap_or("image/jpeg")
        .to_string();

    let data = response
        .bytes()

```

```

        .await
        .map_err(|e| format!("读取图片数据失败: {e}"))?
        .to_vec();

    if data.is_empty() {
        return Err("图片数据为空".to_string());
    }

    Ok(FetchedImage { data, mime })
}

pub async fn save_download_bytes(data: Vec<u8>, dest_path: String) -> Result<String, String> {
    if data.is_empty() {
        return Err("下载数据为空".to_string());
    }

    let dest = path_validator::validate_path(&dest_path, None)?;
    if let Some(parent) = dest.parent() {
        tokio::fs::create_dir_all(parent)
            .await
            .map_err(|e| format!("创建下载目录失败: {e}"))?;
    }

    tokio::fs::write(&dest, &data)
        .await
        .map_err(|e| format!("写入文件失败: {e}"))?;
    Ok(dest.to_string_lossy().to_string())
}

pub async fn embed_audio_metadata(request: EmbedMetadataRequest) -> Result<(), String> {
    let request = request.clone();
    tokio::task::spawn_blocking(move || write_metadata_to_file(&request))
        .await
        .map_err(|e| format!("元数据嵌入任务失败: {e}"))?
}

#[derive(Debug, serde::Deserialize, Default)]
#[serde(rename_all = "camelCase")]
pub struct FinalizeDownloadExtrasRequest {
    pub lyrics_text: Option<String>,
    pub lyrics_path: Option<String>,
    pub cover_url: Option<String>,
    pub cover_path: Option<String>,
    pub metadata: Option<EmbedMetadataRequest>,
    pub embed_cover: bool,
}

#[derive(Debug, Serialize, Default)]
pub struct FinalizeDownloadExtrasResult {
    pub lyrics_saved: bool,
    pub cover_saved: bool,
    pub metadata_embedded: bool,
}

```

```

    pub metadata_error: Option<String>,
    pub cover_data: Option<Vec<u8>>,
    pub cover_mime: String,
}

pub async fn finalize_download_extras(
    request: FinalizeDownloadExtrasRequest,
) -> Result<FinalizeDownloadExtrasResult, String> {
    let mut result = FinalizeDownloadExtrasResult::default();

    if let (Some(text), Some(path)) = (&request.lyrics_text, &request.lyrics_path) {
        if !text.is_empty() {
            let dest = PathBuf::from(path);
            if let Some(parent) = dest.parent() {
                let _ = tokio::fs::create_dir_all(parent).await;
            }
            match tokio::fs::write(&dest, text).await {
                Ok(_) => result.lyrics_saved = true,
                Err(_) => {}
            }
        }
    }

    if let Some(url) = &request.cover_url {
        if !url.is_empty() && (url.starts_with("http://") || url.starts_with("https://")) {
            match fetch_image_bytes(url.clone()).await {
                Ok(img) => {
                    if let Some(path) = &request.cover_path {
                        let actual_ext = if img.mime.contains("png") {
                            ".png"
                        } else {
                            ".jpg"
                        };
                    };
                    let final_path = if path.ends_with(".jpg") && actual_ext == ".png" {
                        format!("{}", path[..path.len() - 4])
                    } else if path.ends_with(".png") && actual_ext == ".jpg" {
                        format!("{}", path[..path.len() - 4])
                    } else {
                        path.clone()
                    };
                    let dest = PathBuf::from(&final_path);
                    if let Some(parent) = dest.parent() {
                        let _ = tokio::fs::create_dir_all(parent).await;
                    }
                    match tokio::fs::write(&dest, &img.data).await {
                        Ok(_) => {
                            result.cover_saved = true;
                        }
                        Err(_) => {}
                    }
                }
            }
        }
    }
}

```

```

    }

    result.cover_data = Some(img.data);
    result.cover_mime = img.mime;
}
Err(_) => {}
}
}

if let Some(mut meta) = request.metadata {
    if request.embed_cover && meta.cover_data.is_none() {
        if let Some(data) = &result.cover_data {
            meta.cover_data = Some(data.clone());
            meta.cover_mime = Some(result.cover_mime.clone());
        }
    }
    let meta = meta.clone();
    match tokio::task::spawn_blocking(move || write_metadata_to_file(&meta)).await {
        Ok(Ok(())) => result.metadata_embedded = true,
        Ok(Err(e)) => {
            result.metadata_error = Some(e);
        }
        Err(e) => {
            result.metadata_error = Some(format!("元数据嵌入任务失败: {e}"));
        }
    }
}

Ok(result)
}

fn download_history_path(data_dir: &Path) -> Result<PathBuf, String> {
    Ok(data_dir.join(DOWNLOAD_HISTORY_FILE))
}

pub async fn read_download_history(data_dir: &Path) -> Result<String, String> {
    let path = download_history_path(data_dir)?;
    if !path.is_file() {
        return Ok("{}".to_string());
    }
    match tokio::fs::read_to_string(&path).await {
        Ok(content) if !content.trim().is_empty() => Ok(content),
        Ok(_) => Ok("{}".to_string()),
        Err(e) => Err(format!("读取下载记录失败: {e}")),
    }
}

pub async fn write_download_history(data_dir: &Path, content: String) -> Result<(), String> {
    let path = download_history_path(data_dir)?;
    if let Some(parent) = path.parent() {

```

```

        tokio::fs::create_dir_all(parent)
            .await
            .map_err(|e| format!("创建下载记录目录失败: {e}"))?;
    }
    tokio::fs::write(&path, content)
        .await
        .map_err(|e| format!("写入下载记录失败: {e}"))?;
    Ok(())
}

#[derive(Debug, Clone, serde::Serialize)]
pub struct ProbeUrlInfo {
    pub url: String,
    pub size: u64,
    #[serde(skip_serializing_if = "Option::is_none")]
    pub error: Option<String>,
}

pub async fn probe_url_size(url: String) -> Result<ProbeUrlInfo, String> {
    if !(url.starts_with("http://") || url.starts_with("https://")) {
        return Err("无效的探测链接".to_string());
    }

    crate::security::ssrf::validate_outbound_url(&url)
        .await
        .map_err(|e| format!("探测链接校验失败: {e}"))?;

    let client = reqwest::Client::builder()
        .timeout(std::time::Duration::from_secs(8))

        .redirect(crate::security::ssrf::ssrf_redirect_policy())

        .dns_resolver(crate::security::ssrf::pinned_dns_resolver())
        .user_agent("Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko
) Chrome/120.0.0.0 Safari/537.36")
        .build()
        .map_err(|e| format!("创建探测客户端失败: {e}"))?;

    let resp = match client
        .get(&url)
        .header(reqwest::header::RANGE, "bytes=0-0")
        .header(
            reqwest::header::ACCEPT,
            "audio/webm, audio/ogg, audio/wav, audio/*;q=0.9, */*;q=0.5",
        )
        .send()
        .await
    {
        Ok(r) => r,
        Err(e) => {
            return Ok(ProbeUrlInfo {

```

```

        url,
        size: 0,
        error: Some(format!("请求失败: {e}")),
    });
}
};

let final_url = resp.url().to_string();
let status = resp.status();

if status == request::StatusCode::PARTIAL_CONTENT {
    let size = resp
        .headers()
        .get(request::header::CONTENT_RANGE)
        .and_then(|v| v.to_str().ok())
        .and_then(|v| v.rsplit('/').next())
        .and_then(|v| v.parse::<u64>().ok())
        .unwrap_or(0);
    return Ok(ProbeUrlInfo {
        url: final_url,
        size,
        error: None,
    });
}

if status.is_success() {
    let size = resp
        .headers()
        .get(request::header::CONTENT_LENGTH)
        .and_then(|v| v.to_str().ok())
        .and_then(|v| v.trim().parse::<u64>().ok())
        .unwrap_or(0);
    return Ok(ProbeUrlInfo {
        url: final_url,
        size,
        error: None,
    });
}

Ok(ProbeUrlInfo {
    url: final_url,
    size: 0,
    error: Some(format!("HTTP {status}")),
})
}

pub async fn fetch_announcement() -> Result<String, String> {
    let url = "https://xy.zh2026.cn/chaoguan/public/api/app.php?action=app_announcement";

    let client = request::Client::builder()
        .timeout(std::time::Duration::from_secs(15))

```

```

        .dns_resolver(crate::security::ssrf::pinned_dns_resolver())
        .user_agent("XY-Music-Updater")
        .http1_only()
        .build()
        .map_err(|e| format!("创建请求客户端失败: {e}"))?;

let resp = client
    .get(url)
    .header("Accept", "application/json")
    .header("Cache-Control", "no-cache")
    .send()
    .await
    .map_err(|e| format!("请求公告接口失败: {e}"))?;

let status = resp.status();
let text = resp
    .text()
    .await
    .map_err(|e| format!("读取公告数据失败: {e}"))?;

if !status.is_success() {
    let snippet: String = text.chars().take(200).collect();
    return Err(format!("公告接口返回错误状态: {status} | 响应: {snippet}"));
}

match serde_json::from_str::<serde_json::Value>(&text) {
    Ok(v) => {
        let code = v.get("code").and_then(|c| c.as_i64()).unwrap_or(0);
        if code == 200 {
            match v.get("data") {
                Some(d) if !d.is_null() => return Ok(d.to_string()),
                _ => return Ok("{}".to_string()),
            }
        }
        let msg = v
            .get("msg")
            .and_then(|m| m.as_str())
            .unwrap_or("公告接口返回未知错误");
        Err(format!("公告接口返回错误: {msg}"))
    }
    Err(_) => {
        let snippet: String = text.chars().take(200).collect();
        Err(format!("公告数据解析失败, 原始响应: {snippet}"))
    }
}

pub async fn write_state_json(data_dir: &Path, key: String, value: String) -> Result<(), String> {
    let sanitized_key = path_validator::sanitize_filename_component(&key)

```

```

        .map_err(|e| format!("无效的 key: {}", e))?;
let state_dir = data_dir.join("state");
tokio::fs::create_dir_all(&state_dir)

        .await
        .map_err(|e| format!("创建 state 目录失败: {e}"))?;
let file_path = state_dir.join(format!("{sanitized_key}.json"));
tokio::fs::write(&file_path, &value)

        .await
        .map_err(|e| format!("写入 state 文件失败: {e}"))?;
Ok(())
}

pub async fn read_state_json(data_dir: &Path, key: String) -> Result<Option<String>, String> {
    let sanitized_key = path_validator::sanitize_filename_component(&key)
        .map_err(|e| format!("无效的 key: {}", e))?;
    let file_path = data_dir.join("state").join(format!("{sanitized_key}.json"));
    if !file_path.exists() {
        return Ok(None);
    }
    let content = tokio::fs::read_to_string(&file_path)
        .await
        .map_err(|e| format!("读取 state 文件失败: {e}"))?;
    Ok(Some(content))
}

pub async fn download_wallpaper(
    data_dir: &Path,
    url: String,
    filename: String,
) -> Result<String, String> {
    use tokio::fs::File;
    use tokio::io::AsyncWriteExt;

    if !(url.starts_with("http://") || url.starts_with("https://")) {
        return Err("无效的壁纸下载链接".to_string());
    }

    crate::security::ssrf::validate_outbound_url(&url)
        .await
        .map_err(|e| format!("壁纸链接校验失败: {e}"))?;

    let safe_name = std::path::Path::new(&filename)
        .file_name()
        .and_then(|n| n.to_str())
        .unwrap_or("wallpaper.jpg")
        .to_string();
    let safe_name = if std::path::Path::new(&safe_name).extension().is_none() {
        format!("{safe_name}.jpg")
    } else {
        safe_name
    }

```

```

};

let wallpaper_dir = data_dir.join("wallpapers");
tokio::fs::create_dir_all(&wallpaper_dir)
    .await
    .map_err(|e| format!("创建壁纸目录失败: {e}"))?;
let dest_path = wallpaper_dir.join(&safe_name);

let client = reqwest::Client::builder()
    .timeout(std::time::Duration::from_secs(60))

    .redirect(crate::security::ssrf::ssrf_redirect_policy())

    .dns_resolver(crate::security::ssrf::pinned_dns_resolver())
    .user_agent("XY-Music-WallpaperDownloader")
    .build()
    .map_err(|e| format!("创建HTTP客户端失败: {e}"))?;

let mut response = client
    .get(&url)
    .send()
    .await
    .map_err(|e| format!("下载壁纸失败: {e}"))?;
if !response.status().is_success() {
    return Err(format!("下载服务器返回错误状态: {}", response.status()));
}

let mut file = File::create(&dest_path)
    .await
    .map_err(|e| format!("创建文件失败: {e}"))?;
while let Some(chunk) = response
    .chunk()
    .await
    .map_err(|e| format!("读取响应数据失败: {e}"))?
{
    file.write_all(&chunk)
        .await
        .map_err(|e| format!("写入文件失败: {e}"))?;
}

Ok(dest_path.to_string_lossy().to_string())
}

pub async fn delete_wallpaper_file(data_dir: &Path, local_path: String) -> Result<(), String> {
    let wallpaper_dir = data_dir.join("wallpapers");
    let target = PathBuf::from(&local_path);

    if !target.exists() {
        return Ok(());
    }
}

```

```

    if !target.is_file() {
        return Err("目标不是可删除的壁纸文件".to_string());
    }

    let canonical_dir = std::fs::canonicalize(&wallpaper_dir)
        .map_err(|e| format!("读取壁纸目录失败: {e}"))?;
    let canonical_target = std::fs::canonicalize(&target)
        .map_err(|e| format!("读取壁纸文件失败: {e}"))?;
    if !canonical_target.starts_with(&canonical_dir) {
        return Err("只能删除应用壁纸目录中的文件".to_string());
    }

    tokio::fs::remove_file(&target)
        .await
        .map_err(|e| format!("删除壁纸文件失败: {e}"))?;
    Ok(())
}

#[cfg(test)]
mod tests {
    use super::FinalizeDownloadExtrasRequest;

    #[test]
    fn finalize_download_extras_request_accepts_frontend_camel_case_payload() {
        let json = serde_json::json!({
            "lyricsText": "[00:00.00]测试歌词",
            "lyricsPath": "/Music/song.lrc",
            "coverUrl": "https://example.com/cover.jpg",
            "coverPath": "/Music/song.jpg",
            "embedCover": true,
            "metadata": {
                "filePath": "/Music/song.mp3",
                "title": "测试歌曲",
                "albumArtist": "测试专辑艺术家",
                "trackNumber": "7",
                "coverMime": "image/jpeg"
            }
        });

        let request: FinalizeDownloadExtrasRequest =
            serde_json::from_value(json).expect("frontend payload should deserialize");

        assert_eq!(request.lyrics_text.as_deref(), Some("[00:00.00]测试歌词"));
        assert_eq!(request.lyrics_path.as_deref(), Some("/Music/song.lrc"));
        assert_eq!(request.cover_url.as_deref(), Some("https://example.com/cover.jpg"));
        assert_eq!(request.cover_path.as_deref(), Some("/Music/song.jpg"));
        assert!(request.embed_cover);

        let metadata = request.metadata.expect("metadata should deserialize");
        assert_eq!(metadata.file_path, "/Music/song.mp3");
        assert_eq!(metadata.title.as_deref(), Some("测试歌曲"));
        assert_eq!(metadata.album_artist.as_deref(), Some("测试专辑艺术家"));
    }
}

```

```
        assert_eq!(metadata.track_number.as_deref(), Some("7"));
        assert_eq!(metadata.cover_mime.as_deref(), Some("image/jpeg"));
    }
}
```